QEC Decoder Data Notes

Stim data generation, labels, formats and preprocessing for ML decoders: a team reference

Sponsor	Dr. Youngseok Kim, Electrical and Computer Engineering, The Ohio State University
Compiled	October 1, 2026, from the team's working sessions on the Stim data pipeline
Software	Stim 1.16.0, PyMatching 2.4.0, NumPy; companion files `stim_pipeline.py`, `example_config.json`, `preprocess_walkthrough.py`
Citations	[n, section] points to reference n in section 24. [S] marks numbers we computed ourselves; rerun the scripts to reproduce them.

In one paragraph. Stim simulates noisy surface-code experiments and returns, for every shot, a row of detection events (did each stabilizer's result change since the previous round?) and one label bit (did the logical qubit flip?). That is the same form as real hardware data. The model learns to predict the label from the detection events. Before training, reorder each row into a [round, stabilizer] grid using the circuit's detector coordinates, add channels for which slots exist, which are X or Z checks and which row is the final readout, stack shots on their own axis, and evaluate by logical error rate against MWPM on the same shots.

Contents

PART I · CONCEPTS

1. Project context
2. Glossary
3. The running example: a distance-3 surface code
4. Shots, rounds and detectors
5. How Stim produces labelled data, compared with hardware
6. Labels: what 0 and 1 mean
7. Z-basis and X-basis experiments
8. Scope questions

PART II · THE DATA

9. The detection-event matrix
10. Patterns to recognise
11. Reading a real shot
12. Stim result formats and which to use
13. Our pipeline: stim_pipeline.py

PART III · PREPROCESSING FOR ML

14. Overview
15. The twelve steps
16. The assembled input
17. Storing many shots
18. Training on several code distances
19. Class balance, noise levels and the right metric
20. How published ML decoders encode their inputs

PART IV · QUICK REFERENCE

21. Common misconceptions
22. Numbers we measured
23. Stim and PyMatching cheat sheet
24. References

Appendix A · preprocess_walkthrough.py

Appendix B · example_config.json

PART I

Concepts

1. Project context

The project goal is a machine-learning decoder for the surface code that is more accurate than a minimum-weight perfect matching (MWPM) decoder while staying computationally light [1, Project description; 2]. Decoding is a classical data-processing problem: a stream of syndrome bits arrives every round, and the decoder must infer the correction [1, Project description].

Phase	What the brief and meetings ask for
Semester 1	Survey the ML decoding literature; analyse the open-source code bases (Stim, PyMatching, published neural decoders); build a pipeline that generates **labelled syndrome datasets** for the surface code under circuit-level noise; reproduce MWPM as the accuracy baseline [1, Objectives]. Meeting of 9/16: finish the Stim pipeline for training and test data, build MWPM with PyMatching in parallel, set numerical targets once the MWPM baseline exists [2, 9/16/26].
Semester 2	Implement two ML decoders (a feed-forward/CNN decoder, then a recurrent or transformer one); benchmark logical error rate against MWPM on simulated data and on Google's public surface-code data; profile latency against the ~1 µs-per-round budget [1, Objectives].
Current requirement	Beat MWPM accuracy. Latency is not yet a requirement [2, 9/9/26].
Stretch	Compress the model (quantization, pruning) and deploy to an FPGA or embedded accelerator [1, Objectives].

The brief names four challenges, and all of them show up in this document: **rare-event statistics** (logical errors are rare, so benchmarks need large datasets), **degeneracy** (many physical error patterns give the same syndrome, so the learning target must be the logical class, not the physical error), **generalization** across code size and noise level, and **accuracy versus latency** [1, Challenges].

This document covers the data side of semester 1: what Stim produces, what the labels mean, how the output is laid out, and how to turn it into tensors for a model.

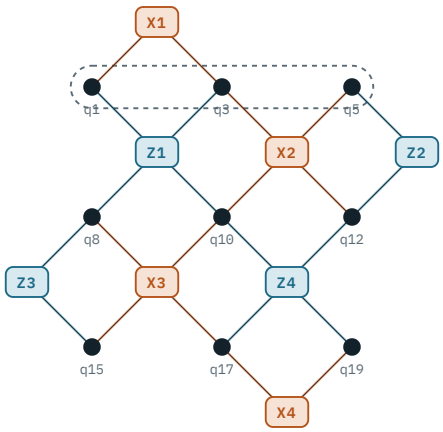
2. Glossary

Term	Meaning	Read more
Data qubit	A physical qubit that holds part of the encoded information. The distance-3 rotated surface code has 9.	[5, §III]
Measurement (ancilla) qubit	An extra qubit used to measure one stabilizer each round; it is measured and reset every round.	[5, §III]
Stabilizer (check)	A parity of 2 or 4 neighbouring data qubits: a product of X operators (X-type) or Z operators (Z-type). Without errors it reads +1 (bit 0). Z-type checks detect X (bit-flip) errors; X-type checks detect Z (phase-flip) errors; a Y error trips both.	[5, §III–V; 6]
Syndrome	The set of stabilizer measurement results in a round.	[6]
Rotated surface code, distance d	d×d data qubits and d^2-1 stabilizers, half X-type and half Z-type. Larger d corrects more errors; below threshold the logical error rate falls as d grows.	[5, §III, VI; 24]
Logical qubit, X_L, Z_L	The encoded qubit. Its logical operators are chains of X or Z across the patch; X_L and Z_L anticommute.	[5, §VI]
Round (QEC cycle)	One pass of measuring every stabilizer once. About 1.1 µs on Google's Willow chip.	[24, abstract]
Shot	One complete, independent run of the experiment: prepare, run all rounds, measure the data qubits.	

Term	Meaning	Read more
Detector	A parity of measurement results that is fixed when there is no noise. In a memory experiment it is mostly the XOR of one stabilizer's results in two consecutive rounds. Declared in Stim with `DETECTOR`.	[7; 9]
Detection event	A detector that reads 1: that stabilizer's result changed since the previous round.	[12, Fig. 1; 14, §4; 4, §2]
Observable	A parity of measurements that equals the logical operator, declared with `OBSERVABLE_INCLUDE`. In a memory experiment it is built from the final data-qubit readout.	[7; 14, §4]
Label	One bit per shot: did the observable come out different from the prepared value?	[14, §4, eq. 18]
Memory experiment	Prepare a logical state, run r rounds of stabilizer measurement, measure all data qubits, check whether the logical value survived. The standard decoder benchmark.	[14, §4]
Basis (X or Z memory)	Which logical state is prepared and read out. Same code, different experiment (section 7).	[14, §4]
Circuit-level noise	Faults after every gate, reset, measurement and idle step, rather than only on data qubits.	[14, §4]
SI1000	Superconducting-inspired circuit noise model: two-qubit gate p, one-qubit gate p/10, initialization 2p, measurement 5p, idle p/10, idle during measurement/reset 2p.	[22, Table 3]
Detector error model (DEM)	A list of independent error mechanisms, each with a probability and the detectors and observables it flips. MWPM uses it as its prior; Stim derives it from a noisy circuit.	[7; 4, §1; 28]
MWPM, PyMatching	Minimum-weight perfect matching: pairs up detection events along the most likely error chains. PyMatching is the standard implementation and our baseline.	[10; 11; 29]
Logical error rate (LER)	Fraction of shots where the decoder's prediction differs from the label; can be converted to a per-round rate.	[14, §4, eq. 20]
Degeneracy	Different physical error patterns can produce the same detection events and the same logical effect, so a decoder targets the logical class, not the exact error.	[14, §2.1.1; 4, §3; 1, Challenges]
Hook error	A single fault on a measurement qubit partway through its check that spreads to several data qubits through the remaining CNOT gates.	[26; 10]
Pauli frame	Corrections are not applied to the qubits; classical software records them and flips later measurement results instead.	[5, §IV–V]
Soft information, leakage	Analog readout values and flags for qubits that left the computational states. Some ML decoders use them; plain Stim does not produce them.	[12, §2.3; 13]

3. The running example: a distance-3 surface code

Every example in this document uses the circuit Stim generates with `stim.Circuit.generated("surface_code:rotated_memory_z", distance=3, rounds=3)` [9]: a Z-basis memory experiment on the rotated surface code with 9 data qubits and 8 stabilizers. Names X1–X4 and Z1–Z4 are ours; `q` numbers are Stim's qubit indices; positions are Stim's coordinates.



Dots are data qubits, boxes are stabilizers (orange X-type, blue Z-type). The final readout of q1, q3 and q5 (dashed box) is the observable: logical Z along one edge.

Check	(x, y)	Ancilla	Data qubits, in CNOT order
X1	(2,0)	q2	q3, q1
Z1	(2,2)	q9	q10, q3, q8, q1
X2	(4,2)	q11	q12, q10, q5, q3
Z2	(6,2)	q13	q12, q5
Z3	(0,4)	q14	q15, q8
X3	(2,4)	q16	q17, q15, q10, q8
Z4	(4,4)	q18	q19, q12, q17, q10
X4	(4,6)	q25	q19, q17

One round is: Hadamard on the X-type ancillas, four layers of CNOTs, Hadamard again, then measure-and-reset (`MR`) of all 8 ancillas. Each `TICK` marks one layer, so a round is about 7 layers. X-type ancillas are the *control* of their CNOTs; Z-type ancillas are the *target*. The CNOT order matters for hook errors (section 11).

4. Shots, rounds and detectors

```
dataset   = many shots                          e.g. 100,000 rows
  shot    = one complete, independent run      one row, one label
    round = one pass measuring every stabilizer  a time step inside the shot
      detector = one stabilizer: "did my result change since last round?"
```

Term	Video analogy	Classical-computer analogy
Gate layer (`TICK`)	–	one clock cycle
Round	one frame	one pass of a check loop (about 7 gate layers), like a DRAM refresh cycle that exists only to stop stored data from degrading
Detector	one pixel that lights when something changed since the previous frame	–
Shot	one clip	one complete program run from reset
Label	one tag per clip: “did it end wrong?”	–

- **Rounds are time.** Rounds within a shot happen in order and depend on each other. Never shuffle them. On Willow one round takes 1.1 µs [24, abstract]; the brief’s ~1 µs per round decoding budget comes from this [1, Objectives; 27].
- **Shots are independent.** Each starts from a fresh reset with new random errors; nothing carries over. On hardware the qubits are re-initialized between shots. Shuffle shots freely.
- **A detector compares two measurements** of the same stabilizer in consecutive rounds. The number of detectors per shot for a rotated memory experiment is $r \times (d^2-1)$: 24 for d = 3 with 3 rounds, 120 for d = 5 with 5 rounds [S].
- **Why several rounds:** a data-qubit error and a measurement error can look identical in one round and only differ in the next (section 10). Telling them apart needs comparisons over time, so experiments run many rounds (our pipeline defaults to rounds = d; Google’s Sycamore data used up to 25) [5, §XI; 12, §2.2; 14, §4].

5. How Stim produces labelled data, compared with hardware

1. The circuit contains noise instructions (`DEPOLARIZE1`, `DEPOLARIZE2`, `X_ERROR`, ...). For each shot Stim draws random Pauli faults from them [7].
2. It simulates the circuit and produces the measurement results a device would produce.
3. `DETECTOR` lines XOR measurement results into detection events.
4. `OBSERVABLE_INCLUDE` lines XOR the final data-qubit readouts into the logical observable. Because the prepared logical state is known, a flip of the observable is the label [7; 14, §4].

A real experiment works the same way: the device gives raw measurement bits; detection events and the label are computed in software from the same definitions, and the label is available only because the experimenters know what they prepared. That is why Stim output and the public Google datasets have the same form: detection events in, one observable-flip bit out [12; 23; 24].

> **Train on the observable flip, not on the Pauli errors.** Hardware can never tell you which physical errors happened, and many different errors are equivalent (degeneracy), so a model trained on physical errors could not be fine-tuned on real data [1, Challenges; 14, §2.1.1]. Stim can report the sampled error mechanisms (`dem.compile_sampler().sample(n, return_errors=True)`), which is useful for debugging and analysis, as in section 11 [9].

Two equivalent ways to sample

```
# Direct: detection events and labels
dets, obs = circuit.compile_detector_sampler(seed=1).sample(100_000, separate_observables=True)

# Hardware-faithful: raw measurements first, then convert exactly as an experiment would
meas = circuit.compile_sampler(seed=1).sample(100_000)
dets, obs = circuit.compile_m2d_converter().convert(measurements=meas, separate_observables=True)
```

What Stim cannot simulate

Stim simulates Clifford circuits with Pauli noise. It does not model leakage, coherent errors, drift or analog (I/Q) readout, which is what separates real data from simulation. The standard recipe is to pretrain on large amounts of simulated data (SI1000, or a DEM fitted to the device) and fine-tune on a smaller amount of real data [12, §2.2; 3, §2.1].

Public hardware data

- **Sycamore** (Google 2023), d = 3 and 5: Zenodo 10.5281/zenodo.6804040, 315 MB, with circuits, samples and several decoders' predictions [23; 3, §2.1].
- **Willow** (Google 2025), d = 3, 5, 7: Zenodo 10.5281/zenodo.13273331; the archive `google_105Q_surface_code_d3_d5_d7.zip` (5.7 GB) holds the surface-code memory data. AlphaQubit 2 used 50,000 shots per duration of 10, 30, ..., 250 cycles, for each distance and each basis [13, A.3.5; 3, §2.2].
- Read the README in each archive for the exact file layout before writing a loader.

6. Labels: what 0 and 1 mean

In a Z-basis experiment the logical qubit is prepared as 0. **Label 0**: the final logical readout is still 0. **Label 1**: it reads 1, so a net logical X (or Y) error happened. The decoder's output answers one question, "should the final answer be flipped back?". The flip is applied in classical software (the Pauli frame), not to the qubits [5, §IV–V]. Training uses binary cross-entropy on this bit [14, §4, eqs. 18–19].

- **Label 1 does not mean the error was uncorrectable.** A single X error on q3, which lies on the readout line, gives label 1. MWPM sees the pattern, predicts 1, and is right [S]. Decoder failure means *prediction ≠ label*.
- **The label is per shot, not per round.** Reading the logical value means measuring every data qubit, which ends the experiment, so it happens once.

Four Pauli classes are two bits

Logical class	Has an X part?	Has a Z part?
I	0	0
X	1	0
Z	0	1
Y	1	1

For one logical qubit there are 4 classes; for CSS codes such as the surface code, the problem splits into an $\bar{X}$ task and a $\bar{Z}$ task, each a binary classification [14, §2.1.2]. A Z-basis experiment labels the first bit, an X-basis experiment the second. One real shot cannot give both, because logical X and Z anticommute and measuring one disturbs the other [5, §VI]. The standard approach is one binary label per basis and training on both bases, with two models or one model with a readout per basis (AlphaQubit 2 uses separate readout parameters per basis) [13, A.2.3].

Per-round labels (simulation only)

- AlphaQubit re-ran the simulator with the same seed for increasing numbers of cycles to get a label at every round, and trained these as extra outputs [12, A.4.6].
- AlphaQubit 2 adds “fake endings” (measuring the data qubits at every cycle in simulation) and noiseless observables (Stim `MPP` measurements, with a Bell pair for the other basis) as auxiliary losses. Its held-out test data comes from a simple pipeline with no intermediate labels [13, A.3.3].
- These labels do not exist on hardware, so they are auxiliary only; the main loss stays on the final flip. The hardware-compatible alternative is to run experiments of several lengths (Willow: 10 to 250 cycles) [13, A.3.5].

Degeneracy, and why one bit is the right target

We planted an X error on q1 in one run and on q3 in another. Both gave exactly the same detection pattern (only Z1 fires, in round 1) and the same label, 1 [S]. No decoder can tell them apart, and it does not need to: the fix is the same. A DEM merges mechanisms that flip the same detectors for the same reason [4, §1; 28]. MWPM reconstructs a likely error chain internally but outputs the same single bit: PyMatching’s `decode()` returned `[1]` for this pattern [10; 11; S]. The ML model and MWPM therefore predict the same quantity, which makes the comparison fair.

7. Z-basis and X-basis experiments

There is one surface code: same qubits, same stabilizers, same circuit every round. The basis describes the experiment [14, §4].

	Z-basis memory	X-basis memory
Prepare	logical 0: every data qubit in $\|0\rangle$	logical +: every data qubit in $\|+\rangle$
Read at the end	every data qubit in Z (0/1 bits)	every data qubit in X
Checks in round 0 and final row	Z-type only	X-type only
Label catches	logical bit flips (X part)	logical phase flips (Z part)
Stim task	`surface_code:rotated_memory_z`	`surface_code:rotated_memory_x`

- **No “XZ basis”:** logical X and Z cannot be read on the same shot. Y-basis memory experiments exist [25] but are uncommon and not needed: X and Z experiments together cover all four classes.
- **Train and report both.** In our SI1000 runs MWPM’s error rates in the two bases were nearly identical (section 13), so neither can be skipped. Start with Z at d = 3, then add X.
- **Mixing bases in one model** requires telling the model the basis (a flag channel or a separate output head), because the same pattern means different things in each (step 9).

---

8. Scope questions

Is a memory experiment enough to train a decoder?

For this project, yes. Memory experiments are the dominant benchmark for neural decoders [14, §4]; AlphaQubit, AlphaQubit 2, Varbanov et al. and Lange et al. all train and evaluate on them [12; 13; 16; 15]. Holding the qubit means running the full stabilizer circuit with all its gates and noise every round, which is what every logical qubit in a computer does all the time. Make the data cover both bases, several round counts and several noise levels. Not covered: logical operations between patches, decoding an unbounded stream with sliding windows [14, §5.2], and hardware effects such as leakage and drift [12; 13].

Do we need a separate decoder for each part of a quantum computer?

No. A fault-tolerant quantum computer is not built like a CPU with an ALU and a control unit made of quantum circuitry. Control is classical electronics and software [5, §I–II]. Each logical qubit is a patch of surface code running the same check cycle, whether it is storing a value or taking part in an operation, and the decoder only ever sees detection events. The closest classical analogy is ECC memory: the same error-correcting logic protects every word regardless of what it holds. What does change the decoder: code distance and code family (different detector layouts); the device’s noise (hence fine-tuning on device data [12]); and logical operations between patches, which is still active research [14, §5.2].

The data

9. The detection-event matrix

Stim returns one row per shot and one column per detector, plus one label column (returned separately with `separate_observables=True`) [9]. For the running example each row has 24 bits:

round 0				round 1								round 2								final				label
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	
Z3	Z1	Z4	Z2	X1	Z1	X2	Z2	Z3	X3	Z4	X4	X1	Z1	X2	Z2	Z3	X3	Z4	X4	Z3	Z1	Z4	Z2	obs
0	0	1	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

One real shot (used again in section 11). The three header lines are not in Stim's output; they come from the circuit.

Why round 0 and the final row have only Z checks

- **Round 0.** All data qubits start in 0, so every Z check has a known value (0) and its first measurement can be compared with it. An X check measured on $|0\rangle$ states gives a random result, so there is nothing to compare; its first result becomes the reference for round 1.
- **Final row.** The data qubits are measured as 0/1 bits (a Z-basis readout). The parity of each Z check's data qubits can be recomputed from these bits and compared with the check's last measurement. A Z-basis readout says nothing about X parities, so X checks get no final detector.
- In an X-basis experiment everything swaps: only X checks in round 0 and the final row.

Why the column order looks irregular

- Columns follow the order of the `DETECTOR` lines in the circuit, nothing more [9].
- Rounds 1 and 2 follow the measurement order `MR 2 9 11 13 14 16 18 25`, i.e. qubit numbering: X1 Z1 X2 Z2 Z3 X3 Z4 X4.
- Round 0 and the final row list the Z checks left to right by x position: Z3 Z1 Z4 Z2.
- X and Z checks are interleaved. Another circuit, or Google's hardware circuits, can use another order.
- **Never assume the order.** Map columns with `circuit.get_detector_coordinates()`, which gives each detector's (x, y, t) [9]. Preprocessing step 1 does this.

Col	Round	Check
0	0	Z3
1	0	Z1
2	0	Z4
3	0	Z2
4	1	X1
5	1	Z1
6	1	X2
7	1	Z2
8	1	Z3
9	1	X3
10	1	Z4
11	1	X4
12	2	X1
13	2	Z1
14	2	X2
15	2	Z2
16	2	Z3
17	2	X3
18	2	Z4
19	2	X4
20	final	Z3
21	final	Z1
22	final	Z4
23	final	Z2

10. Patterns to recognise

Shot	Planted cause	Z3 r0	Z1 r0	Z4 r0	Z2 r0	X1 r1	Z1 r1	X2 r1	Z2 r1	Z3 r1	X3 r1	Z4 r1	X4 r1	X1 r2	Z1 r2	X2 r2	Z2 r2	Z3 r2	X3 r2	Z4 r2	X4 r2	Z3 F	Z1 F	Z4 F	Z2 F	Label
0	no error	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
1	X error on q10 (3,3)	0	0	0	0	0	**1**	0	0	0	0	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
2	X error on q3 (3,1)	0	0	0	0	0	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**1**
3	Z1 misread in round 1	0	0	0	0	0	**1**	0	0	0	0	0	0	0	**1**	0	0	0	0	0	0	0	0	0	0	**0**
4	Z error on q10 (3,3)	0	0	0	0	0	0	**1**	0	0	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
5	Y error on q10 (3,3)	0	0	0	0	0	**1**	**1**	0	0	**1**	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
6	random, p = 1%, # 0	0	0	**1**	0	0	0	0	0	**1**	**1**	0	0	0	0	0	0	0	0	**1**	0	0	0	0	0	**0**
7	random, p = 1%, # 1	0	0	0	0	0	0	**1**	**1**	0	0	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
8	random, p = 1%, # 2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**1**	0	0	0	0	0	0	0	**1**	**0**
9	random, p = 1%, # 3	0	0	0	0	0	0	0	0	0	0	**1**	0	0	0	0	0	0	0	**1**	0	0	0	0	**1**	**1**
10	random, p = 1%, # 4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
11	random, p = 1%, # 5	0	0	0	0	0	0	0	0	0	**1**	0	0	**1**	0	**1**	0	0	**1**	0	**1**	0	0	0	0	**0**

Real Stim output for the d = 3, 3-round Z-basis circuit. Shots 0–5 have exactly one planted error each (noiseless circuit plus one `X_ERROR`, `Z_ERROR` or `Y_ERROR` with probability 1, inserted after round 0; the misread is an `X_ERROR` on Z1's ancilla just before its round-1 measurement). Shots 6–11 are random shots at uniform noise p = 1% [S].

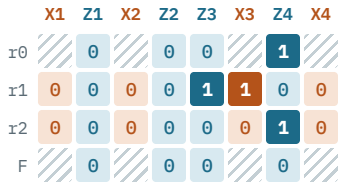
Shot	What the pattern shows
0	nothing fires
1	its two Z neighbours (Z1, Z4) fire once in round 1
2	only Z1 fires: q3 is on the edge with one Z neighbour; q3 is on the readout line, so label 1
3	Z1 fires in round 1 and again in round 2: same check, stacked in time
4	the X checks X2 and X3 fire; Z errors never change a Z-basis label
5	X and Z parts together: Z1, Z4, X2, X3 all fire

Rules of thumb

- **A 1 means "changed since last round"**, not "there is an error here now". An error that happens and stays changes its checks once, so they fire once and then stay at 0 while the error persists.
- **Data-qubit X error:** the neighbouring Z checks fire once, in the same round: two in the bulk, one at an edge. **Z error:** the neighbouring X checks. **Y error:** both kinds at once.
- **Measurement error:** the same check fires in two consecutive rounds, because the wrong reading differs from the reading before it and from the one after it [4, §2, §5.2; 5, §XI].
- **Shots 2 and 3 are identical in round 1 and have opposite labels.** Only round 2 separates a data error on the readout line from a misread. This is why experiments need several rounds and why the decoder must look across time.
- **X checks see different errors from Z checks.** Z checks catch X errors, X checks catch Z errors. A Y error is the X–Z correlation an ML decoder can learn; standard MWPM decodes the X and Z parts separately [14, §2.3; 10].

11. Reading a real shot

The shot from section 9 has events on Z4 in round 0, Z3 and X3 in round 1, and Z4 in round 2; label 0. Laid out as a [round, stabilizer] grid:



The same 24 bits, one row per round. Hatched: no detector.

Check		r0 r1 r2 F
Z4	answer flipped?	1 1 0 0
	event (Stim output)	1 0 1 0
Z3	answer flipped?	0 1 1 1
	event (Stim output)	0 1 0 0
X3	answer flipped?	1 1
	event (Stim output)	1 0

"Answer flipped?" is the running XOR of the events down each column (preprocessing step 10). Events mark the changes in it.

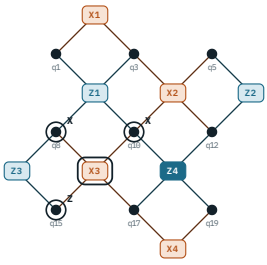
- **Z4: 1 in r0, 0 in r1, 1 in r2.** Z4's answer was already flipped at its first check (a change from the expected 0), stayed flipped in round 1 (no change, so 0), then went back to normal in round 2 (a change again).
- **Z3 and X3: 1 in r1 only.** Both changed once, between round 0 and round 1, and stayed changed. Z3 is still flipped at the end; its final-row detector reads 0 because the final readout agrees with its round-2 check. X3 has no final detector in a Z-basis run.
- **Z1 never fires** although it touches both q8 and q10: two flips on its qubits cancel.

What caused it

The pattern alone does not determine the cause. We sampled Stim's error model for this circuit 10 million times, recording which error mechanisms fired (`return_errors=True`); 101 runs produced exactly this pattern [S]:

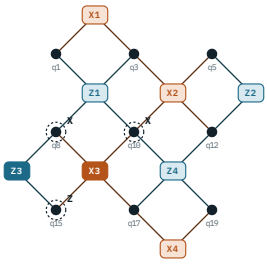
Share	Cause
about 6 in 10	A single two-qubit gate fault on X3's measurement qubit (q16) partway through round 0. X3's ancilla controls CNOTs to q17, q15, q10 and q8 in that order, and an X component on the ancilla is copied onto every data qubit it touches afterwards, so the fault ends up as bit flips on q10 and q8. Its other component either flips X3's own round-0 reading or leaves a phase flip on q15; either way X3 fires in round 1. This spreading is a **hook error** [26; 10]. Z4 checks q10 in the last CNOT layer, after the damage, so it fires in round 0; Z3 checked q8 in the second layer, before it, so it fires in round 1. Then a separate bit flip on q17 or q19 (edge qubits that only Z4 checks) between rounds 1 and 2 restores Z4. The pictures below show the variant with a phase flip on q15.
about 1 in 14	Three separate errors: an early bit flip next to Z4, a Y error on q15 or q8 (lights Z3 and X3 together), and the later flip that restores Z4.
about 1 in 20	Z4 misread in round 0 and again in round 1 (the two round-1 events cancel), plus the Y error on q15 or q8.
the rest	Other combinations of two to four errors.

r0



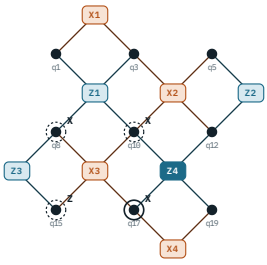
Z4 fires: it checks q10 at the end of round 0, after the fault spread there.

r1



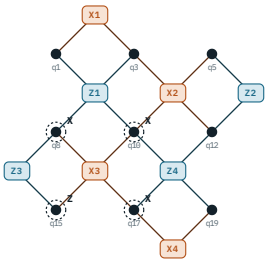
Z3 and X3 fire: their round-0 checks of q8 and q15 came before the damage.

r2



Z4 fires again: a new bit flip on q17 restores its parity.

F



Nothing fires: the final readout agrees with round 2.

Lit boxes: this shot's detection events per round. Rings: data-qubit errors in the most likely explanation (solid: new this round; dashed: still present). The outlined X3 marks the fault location.

- **What the decoder should say.** In 125 full-circuit runs with exactly this pattern the label was 1 only 4 times (about 3%), so the right prediction is 0. None of the likely causes touch the readout line q1, q3, q5 [S].
- **What MWPM says.** PyMatching predicts 0. It pairs Z4-r0 with Z3-r1 and matches X3-r1 and Z4-r2 to the boundary [S]. Because it handles X and Z checks as separate problems, it cannot use the fact that Z3 and X3 lit up together [14, §2.3].
- **Reproduce it:** `circuit.detector_error_model()`, then `dem.compile_sampler().sample(n, return_errors=True)`; map an error index to circuit locations with `circuit.explain_detector_error_model_errors(dem_filter=...)` [9].

12. Stim result formats and which to use

None of the six formats carries headers, coordinates or stabilizer types; all of them load into the same 0/1 array [8]. The format does not change what a model can learn. The structure the model needs (which bit is which stabilizer in which round) comes only from the circuit.

Format	Kind	Layout	Use it for
`01`	dense text	one character per bit, one line per shot	eyeballing small samples; label files
`b8`	dense binary	bits packed 8 per byte (first bit = 1s bit), each shot padded to a byte boundary; reader must know bits per shot	**default for storing detection events**
`r8`	sparse binary	run-length encoded gaps between 1s	very sparse data when disk is tight
`hits`	sparse text	comma-separated indices of the 1 bits	quick inspection of sparse shots
`dets`	sparse text	lines like `shot D5 D12 L0` (D = detector, L = observable, M = measurement)	human-readable sparse records
`ptb64`	dense binary, transposed	64 shots bit-interleaved for SIMD; shot count must be a multiple of 64	high-throughput tooling; skip for ML

- **Generating data inside the training script:** no file format; use the numpy arrays from the sampler directly. With `bit_packed=True` the sampler returns packed uint8 rows [9].
- **Saving to share or reuse:** detection events as **b8**, labels as **01** (one bit per shot, so compression gains nothing and the text is readable), and always the **.stim circuit** beside them: b8 has no header, and the circuit supplies both the bit count and the column meaning.
- **At AlphaQubit scale** (up to ~2×10^9 training samples) data is generated on the fly rather than stored [12; 3, §2.1].

```
import stim
circuit = stim.Circuit.from_file("circuit_noisy.stim")
dets = stim.read_shot_data_file(path="detection_events.b8", format="b8",
                                num_detectors=circuit.num_detectors)       # bool [shots, num_detectors]
obs  = stim.read_shot_data_file(path="obs_flips_actual.01", format="01",
                                num_observables=circuit.num_observables)   # bool [shots, 1]
stim.write_shot_data_file(data=dets, path="out.b8", format="b8", num_detectors=circuit.num_detectors)
```

Tested round trip: 100,000 d = 5, 5-round shots are 1.5 MB as b8 and reload bit-for-bit identical [S]. File names here are the ones our pipeline writes.

13. Our pipeline: `stim_pipeline.py`

A config-driven generator. Everything device-specific lives in a JSON config; the code stays the same. Run with `python stim_pipeline.py --config example_config.json` (Appendix B).

What each experiment folder contains

- `data.npz`: `detection_events` [shots, detectors], `obs_flips` [shots, 1], `grid` [shots, rounds+1, stabilizers], `grid_mask`, `stab_coords`, `stab_type` (0 = Z, 1 = X), `det_round`, `det_stab`, `raw_measurements`, `mwpm_pred`.
- `circuit_noisy.stim`, `model.dem` (the detector error model), `meta.json` (settings, detection-event density, label rate, MWPM logical error rate); optionally `detection_events.b8` and `obs_flips_actual.01`.
- Columns of `grid` are in reading order (top to bottom, then left to right): X1 Z1 X2 Z2 X3 X3 Z4 X4 for d = 3.

Config key	Meaning
`code_task`	Stim family without basis suffix, e.g. `surface_code:rotated_memory`
`bases`	`["Z", "X"]`
`distances, rounds`	e.g. `[3, 5, 7]`; rounds a list or `"d"`
`noise.model`	`none`, `stim_uniform`, `si1000`, or `custom` with separate 1q, 2q, reset, measurement, idle and idle-during-measurement rates
`noise.p, p_sweep`	noise strength, or a list to sweep
`noise.qubit_scale`	per-qubit multipliers for bad qubits, e.g. `{"10": 4.0}`
`circuit_file`	load a device's own noiseless `.stim` circuit and add the chosen noise
`shots, seed`	samples per experiment; seed is offset per experiment
`save_raw_measurements, save_b8, run_mwpm_baseline`	extra outputs

Noise models

- **SI1000** rates from p: two-qubit gate p, one-qubit gate p/10, reset 2p, measurement 5p, idle p/10, idle during a measure/reset layer 2p [22, Table 3]. SI1000 was defined for CZ-based circuits; the pipeline applies the same rates to Stim's CNOT-based circuit, so call it SI1000-style. AlphaQubit 2 used SI1000 at p = 0.15%, giving about 5% detection-event density [13, §3.1].
- **stim_uniform**: Stim's built-in knobs (after-Clifford depolarization, before-round data depolarization, measurement flip, reset flip) all set to p [9].
- Noise is inserted layer by layer (between `TICK`s), including idle noise on qubits not used in a layer.

How X/Z types are inferred

Detectors present in round 0 belong to checks of the experiment's basis (Z checks in a Z-basis run); the rest are the other type. We verified this against which ancillas get Hadamard gates (X-type) for both bases [S]. It assumes a CSS memory experiment whose detectors carry (x, y, t) coordinates, as Stim's and Google's circuits do.

Baseline results (100,000 shots each, SI1000-style noise)

p	d (rounds = d)	MWPM LER, Z basis	MWPM LER, X basis	Label = 1 rate (Z)	Detection-event density
0.001	3	0.29%	0.31%	4.5%	2.4%
0.001	5	0.05%	0.06%	9.7%	2.7%
0.001	7	0.01%	0.01%	16.3%	2.8%
0.003	3	2.30%	2.49%	12.3%	7.0%

p	d (rounds = d)	MWPM LER, Z basis	MWPM LER, X basis	Label = 1 rate (Z)	Detection-event density
0.003	5	1.42%	1.55%	24.0%	7.6%
0.003	7	0.92%	0.95%	34.7%	7.9%

Logical error rate per shot. Error falls with distance at both noise levels, the expected below-threshold behaviour; X and Z bases are nearly identical. The full 12-experiment sweep took about 40 seconds [S].

Limits

Clifford circuits with Pauli noise only (no leakage, coherent errors, drift or soft readout); the automatic layout needs coordinates; another architecture (for example heavy-hex) needs its own circuit file and possibly its own layout function.

PART III

Preprocessing for ML

14. Overview

Each step below shows the data after the step for the shot from section 11, says what the step does and why, and gives the code. The code is `preprocess_walkthrough.py` (Appendix A); its snippets run in order as one script and were tested end to end [S]. The example uses uniform circuit noise at p = 1% so the shot has events to show.

#	Step	Priority	Applies to	Source
1	Reorder into a [round, stabilizer] grid	MUST	round-by-round models (RNN, CNN, transformer)	[9; 12, A.4.1]
2	Add a mask channel	MUST	any model using the grid	[12, A.4.1]
3	Flag the final round	MUST	all	[12, A.4.1; 14, §3.1]
4	Convert to float32; keep labels separate	MUST	all	
5	Split by seed; shuffle whole shots	MUST	all	[13, A.3.3]
6	Add a stabilizer-type channel	SHOULD	all; essential across sizes	[14, §3.2; 12, A.4.3]
7	Lay each round out as a 2D image	SHOULD	CNN only	[12, A.4.2; 19; 20]
8	Train on a mix of noise levels	SHOULD	training set	[14, §2.1.3; 21; S]
9	Add a basis flag	SHOULD	only when one model sees both bases	[13, A.2.3]
10	Add a cumulative-XOR channel	OPTIONAL	all	[12, §2.1, A.4.1]
11	Drop empty shots from training	OPTIONAL	training set only	[15; 14, §2.1.3]
12	Vary the number of rounds	OPTIONAL	recurrent models	[12, Fig. 4; 13, §3]

Skip: normalization or standardization (inputs are already 0/1); deduplicating identical shots (repeats are the real distribution); class re-weighting (labels are 5–35% ones in our data; step 8 handles the shortage of hard examples); symmetry augmentation by flipping or rotating the lattice. Papers use symmetries [14, §2.1.3; 31; 32], but Stim's CNOT ordering breaks some of them, so it is easy to get subtly wrong.

Setup used by all snippets

```
def make_circuit(p=0.01, basis="z", rounds=3, d=3):
    return stim.Circuit.generated(
        f"surface_code:rotated_memory_{basis}", distance=d, rounds=rounds,
        after_clifford_depolarization=p, before_measure_flip_probability=p,
        after_reset_flip_probability=p, before_round_data_depolarization=p)

circuit = make_circuit()
dets, labels = circuit.compile_detector_sampler(seed=11).sample(6, separate_observables=True)
# dets: bool [shots, 24]   labels: bool [shots, 1]
```

15. The twelve steps

1 Reorder into a [round, stabilizer] grid MUST

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0		0		0	0		1	
r1	0	0	0	0	1	1	0	0
r2	0	0	0	0	0	0	1	0
F		0		0	0		0	

Shape [4 rounds, 8 stabilizers]. Column Z4 (outlined) now lines up: events in round 0 and round 2 were positions 2 and 18 of the flat row.

What it does. Uses each detector's (x, y, t) from `circuit.get_detector_coordinates()`. The time picks the row; the position picks the column, in one fixed order for every round (reading order: X1 Z1 X2 Z2 Z3 X3 Z4 X4).

Why. Round-by-round models reuse the same weights on every round, so a column must mean the same stabilizer in every row. In Stim's order Z4 is the 3rd value of round 0 but the 7th of rounds 1 and 2. AlphaQubit likewise keeps one representation per stabilizer. A fully connected network on the flat row can skip this step. [9; 12, §2.1, A.4.1]

```
coords = circuit.get_detector_coordinates()               # {detector: [x, y, t]}
xyt = np.array([coords[k] for k in range(circuit.num_detectors)])
times = np.unique(xyt[:, 2])
rnd = np.searchsorted(times, xyt[:, 2])                    # row of each detector
slots = sorted({(x, y) for x, y, _ in xyt}, key=lambda p: (p[1], p[0]))   # reading order
slot = np.array([slots.index((x, y)) for x, y, _ in xyt])   # column of each detector
grid = np.zeros((len(dets), len(times), len(slots)), np.uint8)
grid[:, rnd, slot] = dets                                  # [shots, 4, 8]
```

2 Add a mask channel MUST

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0		0		0	0		1	
r1	0	0	0	0	1	1	0	0
r2	0	0	0	0	0	0	1	0
F		0		0	0		0	

Events (missing slots hatched, stored as 0).

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	1	0	1	1	0	1	0
r1	1	1	1	1	1	1	1	1
r2	1	1	1	1	1	1	1	1
F	0	1	0	1	1	0	1	0

Mask channel.

What it does. A second grid of the same shape: 1 where a detector exists, 0 where it does not. In a Z-basis run the X checks have no detector in round 0 or the final row. Each new channel is another layer on the same 4×8 grid (section 17).

Why. Empty slots are filled with 0, but 0 also means "did not fire". The mask tells the model which zeros are real. AlphaQubit uses a learned "undefined" embedding for the off-basis stabilizers in the final round for the same purpose. [12, A.4.1]

```
mask = np.zeros((len(times), len(slots)), np.uint8)
mask[rnd, slot] = 1                                        # 1 = a detector exists here
```

3 Flag the final round MUST

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	0	0	0	0	0	0	0
r1	0	0	0	0	0	0	0	0
r2	0	0	0	0	0	0	0	0
F	1	1	1	1	1	1	1	1

Final-round flag.

What it does. A channel that is 1 on the last row and 0 elsewhere.

Why. The last row is not measured by the ancillas; it is recomputed from the final data-qubit readout, so its errors behave differently (data-readout errors instead of ancilla misreads). AlphaQubit embeds the final round separately; the LSTM decoders of Baireuther et al. and Varbanov et al. feed it into a separate output head. [12, A.4.1; 14, §3.1; 16; 17]

```
final = np.zeros_like(mask)
final[-1, :] = 1                                    # last row came from data qubits
```

4 Convert to float32; keep labels separate MUST

What it does. Converts Stim's `bool` arrays to `float32` and keeps the labels in their own array.

Why. PyTorch and TensorFlow layers expect floats. Keeping labels out of the inputs prevents the model from seeing the answer. No scaling is needed.

```
X = grid.astype(np.float32)
y = labels.astype(np.float32)
```

5 Split by seed; shuffle whole shots MUST

What it does. Generates train, validation and test sets with separate `sample()` calls and different seeds, then shuffles the order of training shots.

Why. Separate seeds make the sets independent, so the test score measures performance on unseen shots. Only rows move; columns are positions in space and time. Never filter or rebalance the test set: AlphaQubit 2 computes its reported accuracies from a simple held-out pipeline with no extra processing. [13, A.3.3]

```
def sample(seed, n, c=circuit):
    return c.compile_detector_sampler(seed=seed).sample(n, separate_observables=True)
train, val, test = sample(101, 8_000), sample(102, 1_000), sample(103, 1_000)
order = np.random.default_rng(0).permutation(len(train[0]))
train = (train[0][order], train[1][order])                # rows move; columns never do
```

6 Add a stabilizer-type channel SHOULD

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	1	0	1	0	0	1	0	1
r1	1	0	1	0	0	1	0	1
r2	1	0	1	0	0	1	0	1
F	1	0	1	0	0	1	0	1

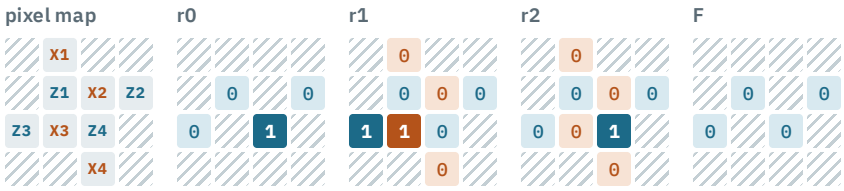
Type channel (1 = X check).

What it does. A constant channel: 1 in X-check columns, 0 in Z-check columns, the same in every row and every shot.

Why. Z checks catch X errors, X checks catch Z errors, and a Y error fires both; knowing the type lets the model learn these relations. Lange et al.'s graph decoder gives every event a one-hot (X, Z) feature; AlphaQubit's attention bias includes a bit for whether two stabilizers share a basis. [14, §3.2; 15; 12, A.4.3]

```
first = {slots[s] for s in slot[rnd == 0]}                # Z-basis: round 0 has only Z checks
is_x = np.array([p not in first for p in slots], np.uint8)  # 1 = X check, 0 = Z check
stype = np.broadcast_to(is_x, mask.shape)
```

7 Lay each round out as a 2D image SHOULD CNN ONLY



What it does. Places each stabilizer at pixel (row, col) = (y/2, x/2) of a (d+1)×(d+1) image: 4×4 for d = 3, 6×6 for d = 5. A shot becomes a short video of shape (4, 4, 4).

Why. Convolutions only look at neighbouring pixels, so physical neighbours must be pixel neighbours. In the step-1 order Z1 and X3 are four columns apart although they share data qubits. AlphaQubit scatters stabilizers onto exactly this grid before its convolutions; 3D convolutional decoders work on the space-time volume. [12, A.4.2; 19; 20]

```
d = 3
img = np.zeros((len(grid), len(times), d + 1, d + 1), np.uint8)
for s, (sx, sy) in enumerate(slots):
    img[:, :, int(sy) // 2, int(sx) // 2] = grid[:, :, s]
```

8 Train on a mix of noise levels SHOULD

Noise p	Label = 1	MWPM wrong	Empty shots
0.1%	2,324 (2.3%)	**77**	84,015 (84.0%)
0.2%	4,496 (4.5%)	**333**	70,841 (70.8%)
0.3%	6,671 (6.7%)	**610**	59,605 (59.6%)
0.5%	10,472 (10.5%)	**1,763**	42,355 (42.4%)
1%	18,680 (18.7%)	**5,830**	18,225 (18.2%)

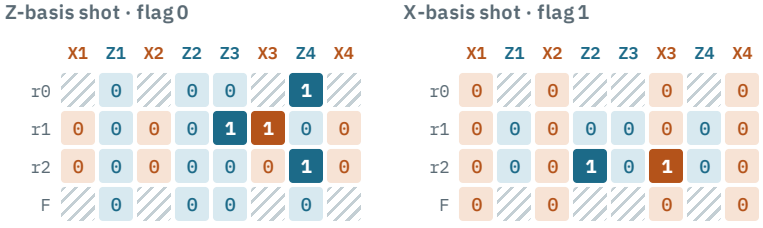
Per 100,000 shots; d = 3, Z basis, 3 rounds, uniform circuit noise [S].

What it does. Builds the training set from several noise levels, then evaluates at the level you care about.

Why. The informative examples are the hard ones, where the obvious answer is wrong. At p = 0.1% there are 77 per 100,000 shots; at 0.5% there are 1,763. Training at elevated noise is a form of importance sampling, studied by Peters. [14, §2.1.3; 21; S]

```
parts = [sample(seed, 2_000, make_circuit(p)) for seed, p in
         enumerate([0.001, 0.002, 0.003, 0.005])]
mix_dets = np.concatenate([a for a, _ in parts])
mix_labels = np.concatenate([b for _, b in parts])
```

9 Add a basis flag SHOULD IF MIXING X AND Z



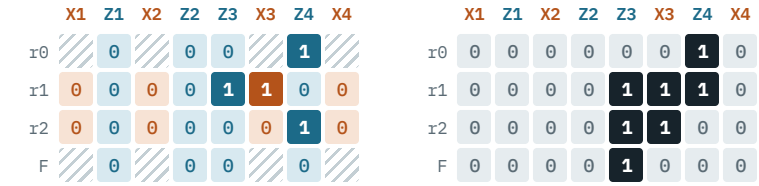
What it does. A constant channel, all 0 for Z-basis shots and all 1 for X-basis shots. Store it as one number per shot.

Why. The same pattern means different things in each basis, and the mask differs (an X-basis run has X checks in round 0 and the final row). AlphaQubit 2 instead uses

separate readout parameters per basis. With two separate models, skip this step. [13, A.2.3]

```
basis_flag = np.zeros(mask.shape, np.uint8)              # all 0 for Z-basis, all 1 for X-basis
```

10 Add a cumulative-XOR channel OPTIONAL



Events. Cumulative XOR (masked).

What it does. Running XOR down each column, set to 0 where no detector exists. For Z checks in a Z-basis run it equals the stabilizer's raw measured value; for X checks, the value relative to its first reading (checked against Stim's raw measurements on 20,000 shots).

Why. Events say "changed"; the running value says "still changed". AlphaQubit trained better and more stably when given measurements as well as events, although they carry the same information. [12, §2.1, A.4.1; S]

```
cumxor = np.bitwise_xor.accumulate(grid, axis=1) * mask     # running XOR down each column;
                                                            # 0 where no detector exists
```

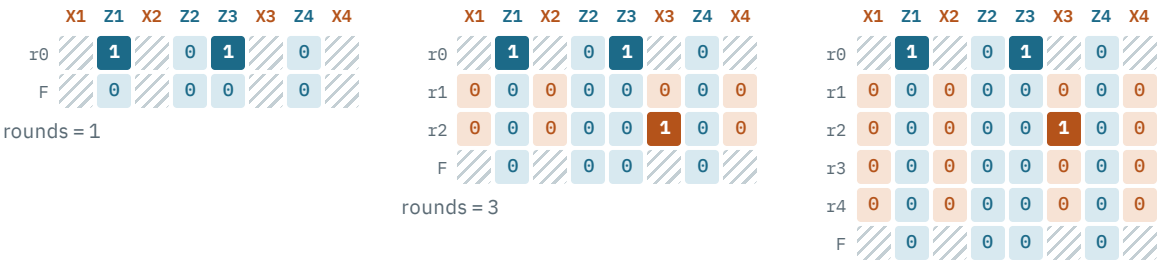
11 Drop empty shots from training OPTIONAL

What it does. Removes shots in which no detector fired, from the training set only. In the 8,000-shot training set at p = 1% this removed 1,464 shots and kept 6,536.

Why. An empty shot almost always has label 0 (about 3 in 100,000 do not, at p = 1%), and at low noise empty shots dominate (84% at p = 0.1%). Lange et al.'s code also filters them. The test set keeps them, because the error rate must be measured over real data. [15; S]

```
keep = train[0].any(axis=1)
train = (train[0][keep], train[1][keep])
```

12 Vary the number of rounds OPTIONAL RNN



What it does. Generates training data at several round counts; grids have r+1 rows.

Why. A recurrent model reads one row at a time and can handle any length, but trained on one length it can overfit to it. AlphaQubit, trained on up to 25 cycles, held its accuracy to 100,000 cycles; AlphaQubit 2, trained on up to 168, was evaluated to one million. Batch shots of the same length together. [12, Fig. 4; 13, §3]

```
circuits_by_rounds = {r: make_circuit(rounds=r) for r in (1, 3, 5)}   # grids with r+1 rows
```

16. The assembled input

channel 0: events

	X1	Z1	X2	Z2	X3	Z4	X4
r0	0	0	0	0	1	0	0
r1	0	0	0	1	1	0	0
r2	0	0	0	0	0	1	0
F	0	0	0	0	0	0	0

channel 1: mask

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	1	0	1	1	0	1	0
r1	1	1	1	1	1	1	1	1
r2	1	1	1	1	1	1	1	1
F	0	1	0	1	1	0	1	0

channel 2: type

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	1	0	1	0	0	1	0	1
r1	1	0	1	0	0	1	0	1
r2	1	0	1	0	0	1	0	1
F	1	0	1	0	0	1	0	1

channel 3: final flag

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	0	0	0	0	0	0	0
r1	0	0	0	0	0	0	0	0
r2	0	0	0	0	0	0	0	0
F	1	1	1	1	1	1	1	1

channel 4: cumulative XOR

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	0	0	0	0	0	1	0
r1	0	0	0	0	1	1	1	0
r2	0	0	0	0	1	1	0	0
F	0	0	0	0	1	0	0	0

Object	Shape
One shot	`[4 rounds, 8 stabilizers, 5 channels]`
Training set	`[N, 4, 8, 5]` inputs and `[N, 1]` labels
CNN version (step 7)	`[N, 4 rounds, 5 channels, 4, 4]`
With a basis flag	6 channels

The model outputs one number, the probability that the label is 1, trained with binary cross-entropy [14, §4, eq. 19].

```
shape = grid.shape
inputs = np.stack([grid,
                   np.broadcast_to(mask, shape),
                   np.broadcast_to(stype, shape),
                   np.broadcast_to(final, shape),
                   cumxor], axis=-1).astype(np.float32)    # [shots, 4, 8, 5]
```

17. Storing many shots: stack, don't append

Right: shots on their own axis

shot 2 · label 0

shot 1 · label 0

shot 0 · label 0

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
r0	0	0	0	0	0	0	1	0
r1	0	0	0	0	1	1	0	0
r2	0	0	0	0	0	0	1	0
F	0	0	0	0	0	0	0	0

Events for N shots: one array [N, 4, 8]; channels add a last axis: [N, 4, 8, 5].

Wrong: shots appended under each other

	X1	Z1	X2	Z2	Z3	X3	Z4	X4
s0·r0	0	0	0	0	0	0	1	0
s0·r1	0	0	0	0	1	1	0	0
s0·r2	0	0	0	0	0	0	1	0
s0·F	0	0	0	0	0	0	0	0
s1·r0	0	0	0	0	0	0	0	0
s1·r1	0	0	1	1	0	0	1	0
s1·r2	0	0	0	0	0	0	0	0
s1·F	0	0	0	0	0	0	0	0

One tall [N×4, 8] table: shot 1's round 0 sits under shot 0's final row as if it were round 4.

- **A 4D array is nested lists:** `X[shot][round][stabilizer][channel]`. Axis 0 = shot, 1 = round (time), 2 = stabilizer (place), 3 = channel. `X[0, 0, 6, 0] = 1` reads: shot 0, round 0, Z4, events. A batch is a slice of axis 0: `X[0:256]` is 256 whole shots.

- **Why not append:** the model can no longer tell where a shot ends; an RNN would run from shot 0 into shot 1 as if time continued, and each label would belong to a block of rows.
- **Do not copy constant channels into every shot.** Mask, type and final flag are identical for every shot of a configuration: store them once and *broadcast* them when building a batch (numpy repeats them along the shot axis without copying) [30]. A basis flag is one number per shot.

Array	Shape	Stored
events	[N, 4, 8] uint8	per shot
labels	[N, 1]	per shot
basis (if mixing)	[N]	per shot, one number
mask, type, final flag	[4, 8] each	once per configuration
cumulative XOR	[B, 4, 8]	computed per batch
model input	[B, 4, 8, 5] float32	built per batch of B shots

Memory: for 1 million d = 5, 5-round shots (120 detectors), the b8 file is 15 MB, the events grid as uint8 [1M, 6, 24] is 144 MB, and a full float32 tensor with 5 channels would be 2.9 GB. Build batches on the fly [S].

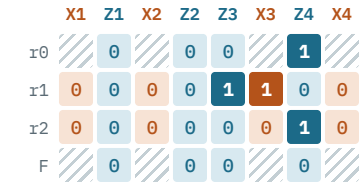
```
events = grid                                          # [N, 4, 8] uint8: the only per-shot input
layout = np.stack([mask, stype, final], axis=-1)       # [4, 8, 3]: same for every shot, store once

def make_batch(idx):
    ev = events[idx]                                   # [B, 4, 8]
    cx = np.bitwise_xor.accumulate(ev, axis=1) * mask  # [B, 4, 8], computed on the fly
    const = np.broadcast_to(layout, ev.shape + (3,))   # [B, 4, 8, 3], a view, no copy
    return np.concatenate([ev[..., None], const, cx[..., None]], axis=-1).astype(np.float32)

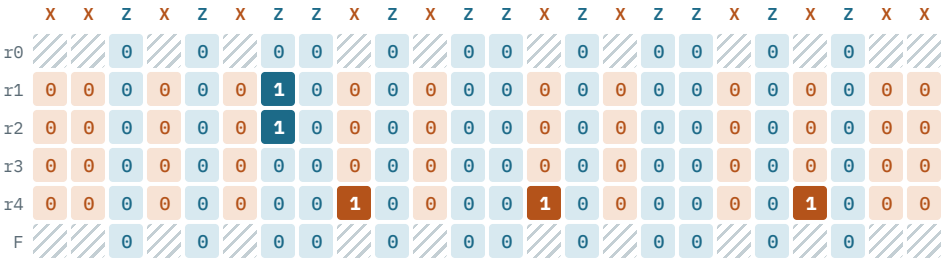
batch = make_batch(np.arange(4))                       # [4, 4, 8, 5] -> straight into the model
```

18. Training on several code distances

d = 3, 3 rounds: [4, 8]



d = 5, 5 rounds: [6, 24]



- **A distance channel does not solve it.** The problem is the shape: 24 stabilizers and 6 rows cannot share an array with 8 and 4, and a model with weights for 8 columns has nowhere to put 24. Once shapes are handled, a distance number as input is harmless but adds nothing the positions and mask do not already say.
- **Option 1, one model per distance.** Simplest; start here. Same pipeline, separate arrays and models for d = 3, 5, 7.
- **Option 2, one model for all distances.** Needs weights that do not depend on the number of stabilizers: a transformer or graph network treating stabilizers as tokens or nodes, or a fully convolutional CNN on the step-7 images. Give each stabilizer its position normalized to the patch as two channels, so edges are always at −0.5 and +0.5. AlphaQubit 2 does this (with rotary position encodings and a curriculum from small to large distances); graph decoders also share weights across distances [13, A.1.2, A.2.4; 14, §3.2].
- **Batches:** keep one distance per batch, or pad smaller shots up to the largest shape with mask = 0 in the padding.

Distance	Raw x	Normalized x = x/(2d) − 0.5
3	0, 2, 4, 6	-0.50, -0.17, +0.17, +0.50
5	0, 2, …, 10	-0.5, -0.3, -0.1, +0.1, +0.3, +0.5

```
def positions(slots, d):
    xy = np.array(slots, dtype=float)                    # (x, y) of each column, 0 .. 2d
    return xy / (2 * d) - 0.5                            # [n_stabilizers, 2], always in [-0.5, 0.5]

pos3 = positions(slots, 3)                               # [8, 2]  for d = 3
```

19. Class balance, noise levels and the right metric

SI1000-style p	d	Shots with label 1	Shots MWPM gets wrong
0.001	3	4.5%	0.29%
0.001	5	9.7%	0.05%
0.001	7	16.3%	0.01%
0.003	3	12.3%	2.30%
0.003	5	24.0%	1.42%
0.003	7	34.7%	0.92%

Z basis, 100,000 shots each [S]. Many label-1 shots are easy: a single error on the readout line flips the label, and MWPM corrects it.

- **Do not judge a model by accuracy.** At d = 5, p = 0.001, a model that always predicts 0 is 90% accurate; a model that is 99.9% accurate is still worse than MWPM's 99.95%. Report the logical error rate next to MWPM's, on the same test shots.
- **Rare events need many shots.** At an error rate of 0.05%, 100,000 shots contain about 50 failures, so the statistical uncertainty is about 14%. Use confidence intervals; AlphaQubit 2 uses Wilson score intervals [1, Challenges; 13].
- **Per-round rate:** for r rounds, $p_{fail}(r) = [1 - (1 - 2p_L)^r]/2 \approx r \cdot p_L$ for small p_L [14, §4, eq. 20].
- **Training noise:** mix noise levels up to about 3–5× the target so hard examples are common, and evaluate at the target (step 8) [14, §2.1.3; 21].

20. How published ML decoders encode their inputs

Decoder	Input	How X/Z and position are encoded	Output	Source
AlphaQubit	Per stabilizer per round: measurements and events (plus I/Q soft values and leakage when available).	Learned per-stabilizer index embedding; separate final-round embedding with one "undefined" embedding for off-basis slots; attention bias from coordinates, offsets, Manhattan distance and a same-basis bit; (d+1)×(d+1) scatter for convolutions.	Logical-flip probability; auxiliary next-stabilizer and per-round labels.	[12, §2.1, A.4.1–A.4.6]
AlphaQubit 2	Measurements and events per stabilizer; 3–6 cycles compressed into one step.	Normalized (x, y) position embedding plus spatial rotary encodings; learned basis embedding for the colour code.	Logical flip per observable; separate readout parameters per basis; auxiliary intermediate labels.	[13, A.1.2, A.2.3, A.3.3]

Decoder	Input	How X/Z and position are encoded	Output	Source
Lange et al. (GNN)	One node per detection event; empty syndromes filtered in training.	Node features (X one-hot, Z one-hot, x, y, t); k-nearest-neighbour edges weighted by inverse distance.	Two heads for the X and Z logical flips.	[15; 14, §3.2]
Baireuther et al.; Varbanov et al. (LSTM)	Per round, a vector of all X and Z defects; final-round defects from the data readout; soft readout in Varbanov et al.	Fixed vector positions per stabilizer.	Logical parity per basis; main head also takes the final-round defects.	[17; 18; 16; 14, §3.1]
Gicev et al. (3D CNN)	Space-time lattice of unit cells with presence bits and boundary information.	Position by lattice placement.	Per-qubit space-like and time-like error bits.	[19]
Gu et al. (3D CNN)	Embedded detection events on the space-time syndrome lattice; depth scaled with d.	3D convolutions; pooled over the support of each logical observable.	Logical-error logits.	[20; 14, §5.3]

The common thread: all of them are built from the same flat detection-event array plus the circuit's coordinates and stabilizer types. The survey by Lee et al. is the map of this design space, organized by architecture family and by the expressivity–scalability–latency trade-off [14; 3, §1.5].

PART IV

Quick reference

21. Common misconceptions

Belief	Correct picture	See
Stim labels each Pauli error, so the model learns which errors happened.	Stim samples random errors internally but the training label is one bit per shot: did the logical observable flip? Physical errors are not observable on hardware and are degenerate.	**§5–6**
A shot is a combination of a few detector measurements.	A shot is one complete run of the experiment. A detector is the XOR of two measurements of one stabilizer in consecutive rounds; a shot contains $r\times(d^2-1)$ of them.	**§4**
Rounds are how many times the circuit runs.	That is shots. Rounds are time steps inside one shot.	**§4**
Each round has its own label.	One label per shot. Per-round labels exist only as simulation-side auxiliary targets.	**§6**
A 1 means there is an error at that stabilizer now.	A 1 means the stabilizer's result changed since the previous round. A lasting error lights its checks once.	**§10–11**
Label 1 means the error was uncorrectable.	Label 1 is the raw logical flip. A decoder fails when its prediction differs from the label.	**§6**
The file format (b8 vs 01) affects what the model can learn.	All formats load into the same 0/1 array. Structure comes from the circuit.	**§12**
Columns are in the same order in every round.	Not in Stim's output. Reorder with detector coordinates.	**§9, step 1**
X-basis and Z-basis are two different codes.	Same code, different experiment (what is prepared and read out).	**§7**
We need one decoder per component of the quantum computer.	No. Control is classical; every logical qubit is the same kind of patch; the decoder only sees detection events.	**§8**
Accuracy is the right metric.	Logical error rate compared with MWPM on the same shots. Always-0 can be 90% accurate.	**§19**
Only about 0.05% of shots have label 1.	That is MWPM's failure rate at $d = 5$, $p = 0.001$. Label 1 occurs in 4–35% of shots in our data.	**§19**
Append the next shot below the previous one.	Stack shots on a new first axis: [N, rounds, stabilizers, channels].	**§17**
Copy the mask and other constant channels into every shot.	Store them once and broadcast at batch time.	**§17**
Add a distance channel to train on several distances.	Distances change the shape; use one model per distance, or a size-independent architecture with normalized positions.	**§18**

22. Numbers we measured

All computed in this project with Stim 1.16.0 and PyMatching 2.4.0 [S]; rerun the scripts to reproduce.

Uniform circuit noise, d = 3, Z basis, 3 rounds (per 100,000 shots)

p	Label 1	MWPM wrong	Empty
0.1%	2,324	77	84,015
0.2%	4,496	333	70,841
0.3%	6,671	610	59,605
0.5%	10,472	1,763	42,355
1%	18,680	5,830	18,225

Uniform noise, 200,000 shots, rounds = d

d	p	Label 1	Empty shots
3	0.1%	2.2%	84.3%
3	0.3%	6.6%	60.0%
3	1%	18.8%	18.2%
5	0.1%	5.7%	42.6%
5	0.3%	15.5%	7.6%
5	1%	35.3%	0.0%

Among empty shots, label 1 occurred in 0 of them at $p \le 0.3\%$ and in about 3 per 100,000 at d = 3, p = 1%.

Other results

- SI1000-style MWPM baseline, both bases, d = 3, 5, 7: section 13.
- Example shot (section 11): 101 of 10,000,000 error-model samples matched its pattern (60 from the hook-error story, 7 from three separate errors, 5 from a double misread); label 1 in 4 of 125 full-circuit matches; PyMatching predicts 0.
- Degeneracy check: X on q1 and X on q3 give identical detection patterns and identical label 1.
- Cumulative XOR equals raw stabilizer values (relative to the first reading for X checks) on all of 20,000 shots.
- Step 11: 1,464 of 8,000 training shots at p = 1% were empty.
- Storage: 100,000 d = 5 shots = 1.5 MB as b8; 1,000,000 = 15 MB b8, 144 MB uint8 grid, 2.9 GB float32 with 5 channels.

23. Stim and PyMatching cheat sheet

```
import stim, pymatching

# Build a circuit
c = stim.Circuit.generated("surface_code:rotated_memory_z", distance=3, rounds=3,
        after_clifford_depolarization=p, before_round_data_depolarization=p,
        before_measure_flip_probability=p, after_reset_flip_probability=p)
c = stim.Circuit.from_file("circuit.stim")          # or load one
c.num_detectors, c.num_observables, c.num_measurements

# Sample
dets, obs = c.compile_detector_sampler(seed=0).sample(n, separate_observables=True)
meas = c.compile_sampler(seed=0).sample(n)                          # raw measurements
dets, obs = c.compile_m2d_converter().convert(measurements=meas, separate_observables=True)

# Layout
coords = c.get_detector_coordinates()               # {detector_index: [x, y, t]}

# Files
stim.write_shot_data_file(data=dets, path="d.b8", format="b8", num_detectors=c.num_detectors)
dets = stim.read_shot_data_file(path="d.b8", format="b8", num_detectors=c.num_detectors)

# Detector error model and MWPM baseline
dem = c.detector_error_model(decompose_errors=True)
m = pymatching.Matching.from_detector_error_model(dem)
pred = m.decode_batch(dets)                         # predicted observable flips, [n, 1]
ler = (pred[:, 0] != obs[:, 0]).mean()
m.decode_to_edges_array(dets[0])                    # which detection events MWPM paired

# Which errors happened (simulation only, for analysis)
D, O, E = c.detector_error_model().compile_sampler(seed=0).sample(n, return_errors=True)
c.explain_detector_error_model_errors(dem_filter=stim.DetectorErrorModel("error(0.1) D2 D8 D9"))
```

Every call above was used in this project **[9; 11; S]**.

24. References

Citations in the text look like [12, A.4.1]: reference 12, section A.4.1. “Project file” means the PDF is in the team’s project folder.

[1] Y. Kim (sponsor), *Machine Learning Decoders for Quantum Error Correction*, project brief, ECE, The Ohio State University. Project file “QEC_ML_Decoder_Project - Prof. Kim.pdf”.

[2] Team 22, Minutes of Meeting, 09/09/26 and 09/16/26. Project files.

[3] Y. Kim, *Machine-Learning Decoders: Study Path, Reproducible Results, and Project Ideas*, project note `decoder_ML.md` (2026-08-25).

[4] Y. Kim, *Detector Error Models: A Summary*, project note `DEM.md` (2026-04-29), summarizing ref. 28. See also `DEM_prior.md` for where DEM probabilities come from.

[5] A. G. Fowler, M. Mariantoni, J. M. Martinis, A. N. Cleland, *Surface codes: Towards practical large-scale quantum computation*, Phys. Rev. A 86, 032324 (2012). arXiv:1208.0928. Project file “surface_code_review.pdf”.

[6] J. Roffe, *Quantum error correction: an introductory guide*, Contemp. Phys. 60, 226 (2019). arXiv:1907.11157.

[7] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum 5, 497 (2021). arXiv:2103.02202. Code: github.com/quantumlib/Stim.

[8] Stim documentation, *Result formats*: github.com/quantumlib/Stim/blob/main/doc/result_formats.md.

[9] Stim documentation, *Python API reference*: github.com/quantumlib/Stim/blob/main/doc/python_api_reference_vDev.md (we used Stim 1.16.0 and its docstrings).

[10] O. Higgott, *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*, arXiv:2105.13082 (2021). Project file.

[11] O. Higgott, C. Gidney, *Sparse blossom: correcting a million errors per core second with minimum-weight matching*, Quantum 9, 1600 (2025). arXiv:2303.15933. Code: github.com/oscarhiggott/PyMatching.

[12] J. Bausch et al., *Learning high-accuracy error decoding for quantum processors* (AlphaQubit), Nature 635, 834 (2024). doi:10.1038/s41586-024-08148-8. Project file "alphaqubit_1.pdf"; section numbers refer to it (A.x = Methods appendix).

[13] A. W. Senior et al., *A scalable and real-time neural decoder for topological quantum codes* (AlphaQubit 2), arXiv:2512.07737v2 (2026). Project file; A.x = appendix sections.

[14] C. Lee, T. Hur, J. Jae, D. K. Park, *Machine Learning Approaches to Decoding Topological Quantum Codes*, arXiv:2608.15760 (2026). Project file. The project's roadmap survey.

[15] M. Lange et al., *Data-driven decoding of quantum error correcting codes using graph neural networks*, Phys. Rev. Research 7, 023181 (2025). arXiv:2307.01241. Code: github.com/LangeMoritz/GNN_decoder (`src/decoder.py`, `src/graph_representation.py`).

[16] B. M. Varbanov, M. Serra-Peralta, D. Byfield, B. M. Terhal, *Neural network decoder for near-term surface-code experiments*, Phys. Rev. Research 7, 013029 (2025). arXiv:2307.03280.

[17] P. Baireuther, T. E. O'Brien, B. Tarasinski, C. W. J. Beenakker, *Machine-learning-assisted correction of correlated qubit errors in a topological code*, Quantum 2, 48 (2018).

[18] P. Baireuther, M. D. Caio, B. Criger, C. W. J. Beenakker, T. E. O'Brien, *Neural network decoder for topological color codes with circuit level noise*, New J. Phys. 21, 013003 (2019).

[19] S. Gicev, L. C. L. Hollenberg, M. Usman, *Fully convolutional 3D neural network decoders for surface codes with syndrome circuit noise*, Quantum Sci. Technol. (2025). arXiv:2506.16113.

[20] A. Gu, J. Ataides, M. D. Lukin, S. F. Yelin, *Scalable neural decoders for practical fault-tolerant quantum computation*, arXiv:2604.08358 (2026).

[21] E. Peters, *Importance sampling for data-driven decoding of quantum error-correcting codes*, arXiv:2505.22741 (2025).

[22] C. Gidney, M. Newman, M. McEwen, *Benchmarking the planar honeycomb code*, Quantum 6, 813 (2022). arXiv:2202.11845. Table 3 defines SI1000.

[23] Google Quantum AI, *Suppressing quantum errors by scaling a surface code logical qubit*, Nature 614, 676 (2023). Data: doi:10.5281/zenodo.6804040.

[24] Google Quantum AI and collaborators, *Quantum error correction below the surface code threshold*, Nature 638, 920 (2025). arXiv:2408.13687. Data: doi:10.5281/zenodo.13273331.

[25] C. Gidney, *Inplace access to the surface code Y basis*, Quantum 8, 1310 (2024). arXiv:2302.07395.

[26] E. Dennis, A. Kitaev, A. Landahl, J. Preskill, *Topological quantum memory*, J. Math. Phys. 43, 4452 (2002). arXiv:quant-ph/0110143. Origin of the term "hook" error.

[27] F. Battistel et al., *Real-time decoding for fault-tolerant quantum computing: progress, challenges and outlook*, Nano Futures 7, 032003 (2023). arXiv:2303.00054.

[28] R. Blume-Kohout, K. Young, *Estimating detector error models from syndrome data*, arXiv:2504.14643 (2025).

[29] A. deMarti iOlius, P. Fuentes, R. Orús, P. M. Crespo, J. Etxezarreta Martinez, *Decoding algorithms for surface codes*, Quantum 8, 1498 (2024). arXiv:2307.14989.

[30] NumPy documentation, *Broadcasting*: numpy.org/doc/stable/user/basics.broadcasting.html.

[31] T. Wagner, H. Kampermann, D. Bruß, *Symmetries for a high-level neural decoder on the toric code*, Phys. Rev. A 102, 042411 (2020).

[32] E. Egorov, R. Bondesan, M. Welling, *The END: An equivariant neural decoder for quantum error correction*, arXiv:2304.07362 (2023).

[S] This project's own simulations: `stim_pipeline.py`, `preprocess_walkthrough.py` and analysis scripts run with Stim 1.16.0 and PyMatching 2.4.0 (September–October 2026). Numbers are statistical estimates from the stated shot counts.

APPENDIX A

preprocess_walkthrough.py

The complete preprocessing script. Every snippet in Part III is a section of it; it runs top to bottom in a few seconds.

```
# Every code snippet shown on the walkthrough page comes from this file, in order.
# Running it end to end checks that the snippets work together.
import stim, numpy as np

# --- setup -------------------------------------------------------------------------
def make_circuit(p=0.01, basis="z", rounds=3, d=3):
    return stim.Circuit.generated(
        f"surface_code:rotated_memory_{basis}", distance=d, rounds=rounds,
        after_clifford_depolarization=p, before_measure_flip_probability=p,
        after_reset_flip_probability=p, before_round_data_depolarization=p)

circuit = make_circuit()
dets, labels = circuit.compile_detector_sampler(seed=11).sample(6, separate_observables=True)
# dets: bool [shots, 24]   labels: bool [shots, 1]

# --- step 1: reorder into a [round, stabilizer] grid -------------------------------
coords = circuit.get_detector_coordinates()               # {detector: [x, y, t]}
xyt = np.array([coords[k] for k in range(circuit.num_detectors)])
times = np.unique(xyt[:, 2])
rnd = np.searchsorted(times, xyt[:, 2])                    # row of each detector
slots = sorted({(x, y) for x, y, _ in xyt}, key=lambda p: (p[1], p[0]))   # reading order
slot = np.array([slots.index((x, y)) for x, y, _ in xyt])   # column of each detector
grid = np.zeros((len(dets), len(times), len(slots)), np.uint8)
grid[:, rnd, slot] = dets                                  # [shots, 4, 8]

# --- step 2: mask ------------------------------------------------------------------
mask = np.zeros((len(times), len(slots)), np.uint8)
mask[rnd, slot] = 1                                        # 1 = a detector exists here

# --- step 3: final-round flag ------------------------------------------------------
final = np.zeros_like(mask)
final[-1, :] = 1                                           # last row came from data qubits

# --- step 4: float32, labels kept separate -----------------------------------------
X = grid.astype(np.float32)
y = labels.astype(np.float32)

# --- step 5: split by seed, shuffle whole shots ------------------------------------
def sample(seed, n, c=circuit):
    return c.compile_detector_sampler(seed=seed).sample(n, separate_observables=True)
train, val, test = sample(101, 8_000), sample(102, 1_000), sample(103, 1_000)
order = np.random.default_rng(0).permutation(len(train[0]))
train = (train[0][order], train[1][order])                # rows move; columns never do

# --- step 6: stabilizer type channel -----------------------------------------------
first = {slots[s] for s in slot[rnd == 0]}                 # Z-basis: round 0 has only Z checks
is_x = np.array([p not in first for p in slots], np.uint8)  # 1 = X check, 0 = Z check
stype = np.broadcast_to(is_x, mask.shape)

# --- step 7: 2D image per round (CNN only) -----------------------------------------
d = 3
img = np.zeros((len(grid), len(times), d + 1, d + 1), np.uint8)
for s, (sx, sy) in enumerate(slots):
    img[:, :, int(sy) // 2, int(sx) // 2] = grid[:, :, s]

# --- step 8: mix noise levels for training -----------------------------------------
parts = [sample(seed, 2_000, make_circuit(p)) for seed, p in
         enumerate([0.001, 0.002, 0.003, 0.005])]
mix_dets = np.concatenate([a for a, _ in parts])
mix_labels = np.concatenate([b for _, b in parts])

# --- step 9: basis flag ------------------------------------------------------------
```

```
basis_flag = np.zeros(mask.shape, np.uint8)                # all 0 for Z-basis, all 1 for X-basis

# --- step 10: cumulative XOR channel ------------------------------------------------------
cumxor = np.bitwise_xor.accumulate(grid, axis=1) * mask     # running XOR down each column;
                                                           # 0 where no detector exists

# --- step 11: drop empty shots (training only) --------------------------------------------
keep = train[0].any(axis=1)
train = (train[0][keep], train[1][keep])

# --- step 12: vary the number of rounds ---------------------------------------------------
circuits_by_rounds = {r: make_circuit(rounds=r) for r in (1, 3, 5)}   # grids with r+1 rows

# --- final: stack channels ----------------------------------------------------------------
shape = grid.shape
inputs = np.stack([grid,
                   np.broadcast_to(mask, shape),
                   np.broadcast_to(stype, shape),
                   np.broadcast_to(final, shape),
                   cumxor], axis=-1).astype(np.float32)    # [shots, 4, 8, 5]

# --- storage: many shots, one array -------------------------------------------------------
events = grid                                              # [N, 4, 8] uint8: the only per-shot input
layout = np.stack([mask, stype, final], axis=-1)           # [4, 8, 3]: same for every shot, store once

def make_batch(idx):
    ev = events[idx]                                       # [B, 4, 8]
    cx = np.bitwise_xor.accumulate(ev, axis=1) * mask      # [B, 4, 8], computed on the fly
    const = np.broadcast_to(layout, ev.shape + (3,))       # [B, 4, 8, 3], a view, no copy
    return np.concatenate([ev[..., None], const, cx[..., None]], axis=-1).astype(np.float32)

batch = make_batch(np.arange(4))                           # [4, 4, 8, 5] -> straight into the model

# --- distances: normalized positions ------------------------------------------------------
def positions(slots, d):
    xy = np.array(slots, dtype=float)                      # (x, y) of each column, 0 .. 2d
    return xy / (2 * d) - 0.5                              # [n_stabilizers, 2], always in [-0.5, 0.5]

pos3 = positions(slots, 3)                                 # [8, 2]  for d = 3
```

APPENDIX B

example_config.json

Config for `stim_pipeline.py`: d = 3, 5, 7, both bases, two noise levels, 100,000 shots each, writing b8/01 files and an MWPM baseline.

```
{
  "out_dir": "stim_data",
  "code_task": "surface_code:rotated_memory",
  "bases": ["Z", "X"],
  "distances": [3, 5, 7],
  "rounds": "d",
  "noise": {
    "model": "si1000",
    "p": 0.001,
    "qubit_scale": {}
  },
  "p_sweep": [0.001, 0.003],
  "shots": 100000,
  "seed": 1234,
  "circuit_file": null,
  "save_raw_measurements": true,
  "save_b8": true,
  "run_mwpm_baseline": true
}
```